

Start coding or [generate](#) with AI.

```
pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
----- 27.9/27.9 MB 57.4 MB/s eta 0:00:00

Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

```
import gensim.downloader as api
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.manifold import TSNE

print("Libraries imported successfully.")
```

Libraries imported successfully.

```
print("Loading word embedding model... This may take a few minutes.")
model = api.load("glove-wiki-gigaword-100")
print("Model loaded successfully.")

vocabulary_size = len(model.index_to_key)
print(f"Vocabulary size: {vocabulary_size}")

# Choose an example word and print its vector
example_word = 'king'
if example_word in model:
    king_vector = model[example_word]
    print(f"Vector for '{example_word}':\n{king_vector[:10]}...") # Print first 10 dimensions for brevity
else:
    print(f"'{example_word}' not found in the vocabulary.")
```

```
Loading word embedding model... This may take a few minutes.
[=====] 100.0% 128.1/128.1MB downloaded
Model loaded successfully.
Vocabulary size: 400000
Vector for 'king':
[-0.32307 -0.87616  0.21977  0.25268  0.22976  0.7388  -0.37954 -0.35307
 -0.84369 -1.1113 ]...
```

```
word_categories = {
    'animals': ['lion', 'tiger', 'elephant', 'giraffe', 'zebra', 'monkey', 'bear', 'wolf'],
    'fruits': ['apple', 'banana', 'orange', 'grape', 'strawberry', 'pineapple', 'mango', 'kiwi'],
    'vehicles': ['car', 'truck', 'bicycle', 'motorcycle', 'airplane', 'boat', 'train', 'bus'],
    'emotions': ['happy', 'sad', 'angry', 'joy', 'fear', 'love', 'surprise', 'calm'],
    'professions': ['doctor', 'teacher', 'engineer', 'artist', 'pilot', 'chef', 'lawyer', 'scientist'],
    'countries': ['usa', 'canada', 'mexico', 'brazil', 'france', 'germany', 'japan', 'china']
}

print("Defined word categories and words:")
for category, words in word_categories.items():
    print(f"- {category.capitalize()}: {' '.join(words)}")
```

```
Defined word categories and words:
- Animals: lion, tiger, elephant, giraffe, zebra, monkey, bear, wolf
- Fruits: apple, banana, orange, grape, strawberry, pineapple, mango, kiwi
- Vehicles: car, truck, bicycle, motorcycle, airplane, boat, train, bus
- Emotions: happy, sad, angry, joy, fear, love, surprise, calm
- Professions: doctor, teacher, engineer, artist, pilot, chef, lawyer, scientist
- Countries: usa, canada, mexico, brazil, france, germany, japan, china
```

```

all_words = []
word_labels = [] # To store categories for plotting later

for category, words in word_categories.items():
    for word in words:
        if word in model:
            all_words.append(word)
            word_labels.append(category)
        else:
            print(f"Warning: '{word}' not found in model vocabulary. Skipping.")

# Extract vectors for the selected words
word_vectors = np.array([model[word] for word in all_words])

print(f"Total words with vectors extracted: {len(all_words)}")
print(f"Shape of word_vectors array: {word_vectors.shape}")

# Display a sample of extracted words, categories, and vector snippets
print("\nSample of extracted words, categories, and vector snippets:")
for i in range(min(5, len(all_words))):
    print(f"Word: {all_words[i]}, Category: {word_labels[i]}, Vector (first 5 dims): {word_vectors[i][:5]}")

```

```

Total words with vectors extracted: 48
Shape of word_vectors array: (48, 100)

```

```

Sample of extracted words, categories, and vector snippets:
Word: lion, Category: animals, Vector (first 5 dims): [ 0.19713  0.43847  0.3363 -0.30903  0.51632]
Word: tiger, Category: animals, Vector (first 5 dims): [-0.82157  0.7952  0.80816 -0.09699 -0.11838]
Word: elephant, Category: animals, Vector (first 5 dims): [-0.066797  0.82425  0.60704 -0.082573 -0.19135 ]
Word: giraffe, Category: animals, Vector (first 5 dims): [ 0.41473  0.31884  0.54587 -0.096985 -0.013342]
Word: zebra, Category: animals, Vector (first 5 dims): [ 0.143 -0.24288  0.24794 -0.26786  0.17935]

```

```

tsne = TSNE(n_components=2, random_state=42, perplexity=10, learning_rate=200)
tsne_results = tsne.fit_transform(word_vectors)

```

```

# Store the results
x_coords = tsne_results[:, 0]
y_coords = tsne_results[:, 1]

print(f"Shape of t-SNE results: {tsne_results.shape}")
print("t-SNE dimensionality reduction complete.")

```

```

Shape of t-SNE results: (48, 2)
t-SNE dimensionality reduction complete.

```

```

plt.figure(figsize=(15, 12))
sns.set_style("whitegrid")

# Create a color palette for the categories
num_categories = len(word_categories)
palette = sns.color_palette("husl", num_categories)
category_colors = {category: palette[i] for i, category in enumerate(word_categories.keys())}

# Plot the t-SNE results
for i, word in enumerate(all_words):
    category = word_labels[i]
    color = category_colors[category]
    plt.scatter(x_coords[i], y_coords[i], color=color, s=80, alpha=0.7)
    plt.annotate(word, (x_coords[i], y_coords[i]), textcoords="offset points", xytext=(5,-5), ha='center', fontsize=9)

# Create custom legend handles
legend_handles = []
for category, color in category_colors.items():
    legend_handles.append(plt.Line2D([0], [0], marker='o', color='w', label=category.capitalize(),
                                     markerfacecolor=color, markersize=10))

plt.legend(handles=legend_handles, title="Categories", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.title('t-SNE Visualization of Word Embeddings by Category', fontsize=16)
plt.xlabel('t-SNE Dimension 1', fontsize=12)

```

```
plt.ylabel('t-SNE Dimension 2', fontsize=12)
plt.grid(True)
plt.tight_layout()
plt.show()
```

